Set Membership Proof

Formal Protocol Specification

Explicit Shuffle, Implicit Vector Commitment, and Hybrid Global Commitment

ZKP AML Three-Entity Protocol Model

Field	Value
Date	3 August 2026
Entities	User Wallet owner, Alipay+ verifier, PayPay prover
Protocols	Explicit Shuffle-Based, Implicit Vector Commitment, Hybrid Global Commitment
Artifact	output/pdf/set_membership_formalization.pdf

1. Commitment Layer and Set Semantics

```
Public parameter:
  pp = (G, g_id, h)

Primary Pedersen ID commitment:
  Com_pp(id; r) = g_id*id + h*r

Transaction-ID commitment:
  c_tid = Com_pp(tid; r_tid)

Rerandomization:
  ReRand_pp(C; rho) = C + h*rho

Object domain:
  X = (x_0, ..., x_{n-1})

Commitment representation:
  S = (C_0, ..., C_{n-1})
  C_i = Com_pp(id_i; r_i)

Set namespace:
  set_id = H(provider_id || policy_id || epoch || root || version)

Optional AML value-binding extension:
  IBC_pp(id, v; r) = g_id*id + g_val*v + h*r

General relation:
  R_member(stmt,w)=1 iff exists i,rho:
    c_tid = ReRand_pp(S[i]; rho)
```

Topic	Formal position
Primary commitment	For an ID-only statement, the primary formal commitment is Pedersen: Com_pp(id; r) = g_id*id + h*r.
IBC extension	IBC is optional for AML statements that also need value or policy binding, for example IBC_pp(id, v; r).
Object set vs commitment set	X is the raw object set. S is the commitment representation used for proofs, so raw transaction or identity objects need not be revealed.
Why implicit/hybrid over commitments	They reduce communication, verification cost, or disclosed subset size while preserving the hidden object layer.
set_id	A public namespace/epoch handle that binds a proof to the intended provider, policy, epoch, root, and version.

2. Function Semantics for Implicit and Hybrid Protocols

The following tables define what the named functions prove. This is the core reason why the implicit and hybrid protocols are meaningful, rather than only a list of abstract function names.

Function	Public input	Witness	Relation proven	Privacy note
VC.Commit(V)	ordered vector V=(V_0,...,V_{n-1})	none	outputs R_V binding every index and value	no membership privacy by itself
VC.Open(V,i)	V and index i	vector auxiliary data	outputs aux_i for element V_i	public opening reveals i
VC.Verify(R_V,i,V_i,aux_i)	R_V, i, V_i, aux_i	none	accepts iff V_i is committed at index i under R_V	verifiable opening only
PermVC.ProvePerm(pp,S,R_V; sigma,Delta,aux_all)	pp, S, R_V	sigma, Delta, openings for V	proves R_V=VC.Commit(V), V_k=ReRand_pp(S[sigma(k)];delta_k), and sigma is a permutation	hides permutation and rerandomization witnesses
PermVC.VerifyPerm(pp,S,R_V,pi_perm_vc)	pp, S, R_V, pi_perm_vc	none	accepts iff R_V commits to a permutation/rerandomization of S	prevents unrelated vector commitment
ZKVCMember.Prove(pp,R_V,c_tid; i,aux_i,rho)	pp, R_V, c_tid	hidden i, aux_i, rho	proves VC.Verify(R_V,i,V_i,aux_i)=1 and c_tid=ReRand_pp(V_i;rho)	keeps i and V_i hidden

Function	Public input	Witness	Relation proven	Privacy note
CommitBucket(B_j)	ordered bucket B_j	none	outputs D_j binding all elements and positions in B_j	bucket privacy depends on proof shape
CommitGlobal(D_0,...,D_{m-1})	bucket roots D_0,...,D_{m-1}	none	outputs R_S binding every bucket root	does not reveal bucket content
ProvePartition(pp,S,R_S; B_0,...,B_{m-1})	pp, S, R_S	buckets and assignment witnesses	proves all bucket elements rerandomize from S, every S element appears exactly once, and R_S=CommitGlobal(CommitBucket(B_0),...,CommitBucket(B_{m-1}))	can hide assignment details in ZK
VerifyPartition(pp,S,R_S,pi_partition)	pp, S, R_S, pi_partition	none	verifies global bucket consistency with S	prevents arbitrary buckets
ProveGlobal/VerifyGlobal	R_S and bucket claim/proof	hidden or public bucket witness	proves the used bucket commitment belongs under R_S	j can be hidden when required
BucketMember.Prove/Verify	bucket/global root and c_tid	hidden member position and rho	proves c_tid is rerandomization-equivalent to a committed bucket member	can hide bucket index and position

3. Protocol 1: Explicit Shuffle-Based Set Membership

Parameter	pp = (G, g_id, h) S = (C_0, ..., C_{n-1}) Shuffle = (Setup, Prove, Verify) ZKLinkAny = (Prove, Verify)

Input	Prover:
	  stmt = (pp, S, c_tid, set_id)
	  w_exp = (i, sigma, Delta, rho, tau)
	  sigma(tau) = i
	  c_tid = ReRand_pp(S[i]; rho)
	
	Verifier:
	  stmt = (pp, S, c_tid, set_id)
	  pi_exp = (C_prime, pi_shuffle, pi_member)

Output	ACCEPT iff c_tid is proven rerandomization-equivalent to one hidden element of S. ABORT if set_id, shuffle, or anonymous member-link verification fails.
Setup	1. Verifier fixes set_id for the active root, epoch, and policy. 2. Prover and verifier agree on pp and committed set S for set_id.

Prover: PayPay	**Verifier: Alipay+**

3. Choose hidden output position tau and permutation sigma
  with sigma(tau)=i.
4. For k=0..n-1:
  C_prime[k] = ReRand_pp(S[sigma(k)]; delta_k)
5. pi_shuffle = Shuffle.Prove(pp,S,C_prime; sigma,Delta)
6. rho_prime = rho - delta_tau.
7. pi_member =
  ZKLinkAny.Prove(pp,C_prime,c_tid; tau,rho_prime)
8. Send (set_id,C_prime,pi_shuffle,pi_member).
	9. Check set_id and |C_prime|=|S|.
	10. b_shuffle =
	  Shuffle.Verify(pp,S,C_prime,pi_shuffle)
	11. b_member =
	  ZKLinkAny.Verify(pp,C_prime,c_tid,pi_member)
	12. ACCEPT iff b_shuffle=1 and b_member=1.

Accept/Abort Condition	ACCEPT iff:
	  set_id is valid
	  AND |C_prime| = |S|
	  AND Shuffle.Verify(pp,S,C_prime,pi_shuffle)=1
	  AND ZKLinkAny.Verify(pp,C_prime,c_tid,pi_member)=1
	
	ABORT otherwise.

Relation	R_exp(stmt,w_exp)=1 iff exists i,tau,sigma,Delta,rho:
	  sigma is a permutation
	  sigma(tau)=i
	  for all k:
	  C_prime[k]=ReRand_pp(S[sigma(k)];delta_k)
	  c_tid=ReRand_pp(S[i];rho)
	  equivalently exists rho_prime:
	  c_tid=ReRand_pp(C_prime[tau];rho_prime)

Security Intuition	Shuffle.Prove hides the original position. ZKLinkAny proves that c_tid links to one hidden shuffled output without revealing i, tau, or the target mapping.

4. Protocol 2: Implicit Vector Commitment Set Membership

Parameter	
	pp S = (C_0, ..., C_{n-1}) VC = (Setup, Commit, Open, Verify) PermVC = (ProvePerm, VerifyPerm) ZKVCMember = (Prove, Verify)

Input	

	Prover:
	  stmt = (pp,S,c_tid,set_id)
	  w_vc = (i,sigma,Delta,rho,aux_i)
	  V[i] = ReRand_pp(S[sigma(i)]; delta_i)
	  c_tid = ReRand_pp(V[i]; rho)
	
	Verifier:
	  pp,set_id,S descriptor,R_V,pi_perm_vc,c_tid,pi_zk_vc

Output	
	ACCEPT iff R_V commits to a permutation/rerandomization of S and c_tid is proven to open at one hidden index of R_V. ABORT if either proof fails.

Setup	

	1. Choose permutation sigma and rerandomization vector Delta.
	2. For k=0..n-1:
	  V[k] = ReRand_pp(S[sigma(k)]; delta_k)
	3. R_V = VC.Commit(V)
	4. pi_perm_vc =
	  PermVC.ProvePerm(pp,S,R_V; sigma,Delta,aux_all)
	5. set_id = H(provider_id || policy_id || epoch || R_V || version)

Prover: PayPay	Verifier: Alipay+

6. Find hidden index i such that:
  c_tid = ReRand_pp(V[i]; rho)
7. pi_zk_vc =
  ZKVCMember.Prove(pp,R_V,c_tid; i,aux_i,rho)
8. Send (set_id,R_V,pi_perm_vc,c_tid,pi_zk_vc).
	9. Check set_id binds to expected R_V and epoch.
	10. b_perm =
	  PermVC.VerifyPerm(pp,S,R_V,pi_perm_vc)
	11. b_member =
	  ZKVCMember.Verify(pp,R_V,c_tid,pi_zk_vc)
	12. ACCEPT iff b_perm=1 and b_member=1.

Accept/Abort Condition	

	ACCEPT iff:
	  set_id is valid for R_V
	  AND PermVC.VerifyPerm(pp,S,R_V,pi_perm_vc)=1
	  AND ZKVCMember.Verify(pp,R_V,c_tid,pi_zk_vc)=1
	
	ABORT otherwise.

Relation	

	R_imp(stmt,w_vc)=1 iff exists V,i,sigma,Delta,rho,aux_i:
	  R_V = VC.Commit(V)
	  for all k:
	  V[k] = ReRand_pp(S[sigma(k)]; delta_k)
	  c_tid = ReRand_pp(V[i]; rho)
	  PermVC.VerifyPerm(pp,S,R_V,pi_perm_vc)=1
	  ZKVCMember.Verify(pp,R_V,c_tid,pi_zk_vc)=1

Security Intuition	
	A public VC opening (i,value,pi_vc) reveals i and is only a verifiable opening. ZKVCMember keeps i hidden, while PermVC prevents committing to an unrelated vector.

5. Protocol 3: Hybrid Global Commitment Set Membership

Parameter	pp S = (C_0, ..., C_{n-1}) m = number of buckets Bucket = (Assign, CommitBucket) Global = (CommitGlobal, VerifyGlobal) Partition = (ProvePartition, VerifyPartition) BucketMember = (Prove, Verify)

Input	Prover:
	  stmt = (pp,S,c_tid,set_id)
	  w_hyb = (j,S_j,sigma_j,Delta_j,rho,w_global,w_member)
	  c_tid is rerandomization-equivalent to one hidden element of S_j
	
	Verifier:
	  pp,set_id,S descriptor,R_S,pi_partition,c_tid,
	  pi_global,pi_bucket_member

Output	ACCEPT iff the committed bucket system is globally consistent with S and c_tid belongs to one hidden committed bucket. ABORT if partition, global, or bucket membership verification fails.

Setup	1. Partition and optionally shuffle/rerandomize S into:
	     B_0, ..., B_{m-1}
	2. For each bucket j:
	     D_j = CommitBucket(B_j)
	3. R_S = CommitGlobal(D_0, ..., D_{m-1})
	4. pi_partition =
	     ProvePartition(pp,S,R_S; B_0,...,B_{m-1})
	5. set_id = H(provider_id || policy_id || epoch || R_S || version)

Prover: PayPay	Verifier: Alipay+

6. Choose hidden bucket j and hidden member tau:
     c_tid = ReRand_pp(B_j[tau]; rho)
7. pi_global =
     ProveGlobal(pp,R_S,D_j; j,w_global)
8. pi_bucket_member =
     BucketMember.Prove(pp,D_j,c_tid; tau,rho,w_member)
9. Send (set_id,R_S,pi_partition,c_tid,
         pi_global,pi_bucket_member).
	10. Check set_id binds to expected R_S and epoch.
	11. b_part =
	      VerifyPartition(pp,S,R_S,pi_partition)
	12. b_global = VerifyGlobal(pp,R_S,pi_global)
	13. b_member =
	      BucketMember.Verify(
	        pp,R_S,c_tid,pi_bucket_member)
	14. ACCEPT iff all three checks equal 1.

Accept/Abort Condition	ACCEPT iff:
	  set_id is valid for R_S
	  AND VerifyPartition(pp,S,R_S,pi_partition)=1
	  AND VerifyGlobal(pp,R_S,pi_global)=1
	  AND BucketMember.Verify(
	       pp,R_S,c_tid,pi_bucket_member)=1
	
	ABORT otherwise.

Relation	R_hyb(stmt,w_hyb)=1 iff exists B_0,...,B_{m-1},j,tau,rho:
	  R_S = CommitGlobal(D_0, ..., D_{m-1})
	  for all j: D_j = CommitBucket(B_j)
	  VerifyPartition(pp,S,R_S,pi_partition)=1
	  VerifyGlobal(pp,R_S,pi_global)=1
	  c_tid = ReRand_pp(B_j[tau];rho)
	  BucketMember.Verify(
	    pp,R_S,c_tid,pi_bucket_member)=1
	
	Implication:
	  VerifyPartition => union(B_j) is a permutation/rerandomization of S
	  VerifyGlobal => hidden bucket commitment belongs under R_S
	  BucketMember => c_tid belongs to that hidden bucket
	  Therefore c_tid belongs to S.

Security Intuition	The partition proof prevents arbitrary bucket construction. The global proof prevents use of a bucket outside R_S. The bucket member proof can hide bucket index j when bucket privacy is required.

6. Stakeholder Feedback Resolution Matrix

No.	Feedback	Formal revision
1	Why IBC instead of a standard commitment?	The primary formal commitment is now Pedersen ID commitment. IBC is only an optional extension for value or policy binding.
2	What is set_id?	set_id is H(provider_id || policy_id || epoch || root || version), binding verification to one provider, policy, epoch, root, and version.
3	Is S a set of objects or commitments?	X is the object set. S is the commitment representation. Implicit and hybrid reduce verification cost or disclosure over committed sets.
4	c_tid is not equal to S[i]; it is Rerand(S[i]).	The relation is changed to exists i,rho: c_tid = ReRand_pp(S[i];rho).
5	sigma(0)=i should not be proved because verifier must not know i.	Explicit protocol uses hidden tau with sigma(tau)=i and verifies ZKLinkAny without revealing i or tau.
6	Public VC opening reveals i and becomes only verifiable opening.	Function semantics now define VC.Open/Verify as public opening only, then define PermVC and ZKVCMember relations for hidden-index membership.
7	Hybrid needs global consistency and bucket privacy details.	Function semantics now define CommitBucket, CommitGlobal, ProvePartition, VerifyGlobal, and hidden bucket membership.